Answer 1

Part 1: Eviction for Object E (3 MB)

The access rate for each object is computed as the number of references divided by the sliding window length (10 s). The eviction scores are then:

Object	Size (MB)	ReadCost (ms)	AccessRate (ref/s)	EvictionScore
A	4	50	$20/10 = 2.0$	$(2.0 \times 50)/4 = 25.0$
B	2	200	$5/10 = 0.5$	$(0.5 \times 200)/2 = 50.0$
C	3	30	$2/10 = 0.2$	$(0.2 \times 30)/3 = 2.0$
D	1	100	$8/10 = 0.8$	$(0.8 \times 100)/1 = 80.0$

The buffer is completely full at $4 + 2 + 3 + 1 = 10$ MB. Object E requires 3 MB, so at least 3 MB must be freed.

Ranking objects by eviction score (lowest first): C (2.0) $<$ A (25.0) $<$ B (50.0) $<$ D (80.0).
Evicting object C alone frees exactly 3 MB, which is sufficient for E. Since C has the lowest eviction score, it is the natural first choice and no further eviction is required.

Answer: Evict object C (eviction score = 2.0, size = 3 MB).

Part 2: Evict A alone vs. C and D together

Both options free 4 MB of buffer space. To compare them fairly, we compute a *combined* eviction score that aggregates the expected re-read cost over the total space freed:

$$\text{CombinedScore} \;=\; \frac{\sum_{i \in S} \big(\text{AccessRate}(i) \times \text{ReadCost}(i)\big)}{\sum_{i \in S} \text{Size}(i)}$$

Option (a) – Evict A alone (frees 4 MB):

$$\text{CombinedScore}(A) \;=\; \frac{2.0 \times 50}{4} \;=\; \frac{100}{4} \;=\; 25.0$$

Option (b) – Evict C and D together (frees $3 + 1 = 4$ MB):

$$\text{CombinedScore}(C{+}D) \;=\; \frac{0.2 \times 30 \;+\; 0.8 \times 100}{3 + 1} \;=\; \frac{6 + 80}{4} \;=\; \frac{86}{4} \;=\; 21.5$$

A lower combined score means the evicted set is less costly to lose per unit of freed space. Since $21.5 < 25.0$, evicting C and D together incurs a lower expected re-read cost per MB than evicting A alone.

Answer: Evicting C and D together (combined score 21.5) is preferable to evicting A alone (score 25.0).

Answer 2

Part 1: Stale FSM entries at T_1

The FSM encoding is: 00 for $[0, 30)\%$, 01 for $[30, 60)\%$, 10 for $[60, 90)\%$, 11 for $[90, 100]\%$.

Block	Recs at T_0	% at T_0	FSM at T_0	Change	Recs at T_1	% at T_1
1	62	62%	10	+5	67	67%
2	85	85%	10	+10	95	95%
3	20	20%	00	+60	80	80%
4	45	45%	01	−10	35	35%
5	75	75%	10	+20	95	95%
6	95	95%	11	—	95	95%

Computing the correct FSM at T_1 and comparing with the stale (unchanged) FSM:

Block	Stale FSM	Correct FSM at T_1	Stale?
1	10	10	No
2	10	11	**Yes**
3	00	10	**Yes**
4	01	01	No
5	10	11	**Yes**
6	11	11	No

Answer: Blocks 2, 3, and 5 have stale FSM entries at T_1.

Part 2: Inserting 30 records using the stale FSM

The system scans the stale FSM in block order, looking for a block whose FSM guarantees enough room for the remaining batch. The guaranteed free slots per FSM category are:

FSM bits	Guaranteed free slots (out of 100)	
00	≥ 70	(block is at most 30% full)
01	≥ 40	(block is at most 60% full)
10	≥ 10	(block is at most 90% full)
11	≥ 0	(no guaranteed free space)

We need to place 30 records. Scanning blocks with the stale FSM:

1. **Block 1** (FSM = 10): guarantees ≥ 10 free. $10 < 30$; skip.
2. **Block 2** (FSM = 10): guarantees ≥ 10 free. $10 < 30$; skip.

3. **Block 3** (FSM = 00): guarantees ≥ 70 free. $70 \geq 30$; **attempt insertion here.**
   Actual occupancy is 80 records, so only 20 slots remain. Insert 20 records; block is now full (100). Remaining: $30 - 20 = 10$ records.
4. **Block 4** (FSM = 01): guarantees ≥ 40 free. $40 \geq 10$; **attempt insertion here.**
   Actual occupancy is 35 records, so 65 slots remain. Insert all 10 remaining records. Block 4 now has 45 records. Remaining: 0.

No new block is allocated. The final occupancies and refreshed FSM are:

Block	Records after insertion	% full	Refreshed FSM
1	67	67%	10
2	95	95%	11
3	100	100%	11
4	45	45%	01
5	95	95%	11
6	95	95%	11

Answer: Final FSM after refresh:
Block 1 = 10, Block 2 = 11, Block 3 = 11, Block 4 = 01, Block 5 = 11, Block 6 = 11.

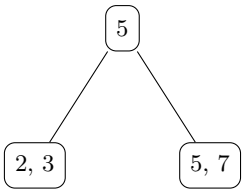
Answer 3

B$^+$ Tree Parameters:

- **Max pointers per node:** $4 \Rightarrow$ **Max keys per node:** 3
- **Leaf minimum occupancy:** $\lceil 3/2 \rceil = 2$ keys
- **Internal node minimum pointers:** $\lceil 4/2 \rceil = 2$ pointers (i.e., at least 1 key), except for the root
- **Pointer convention:** Left pointer $\rightarrow <$; Right pointer $\rightarrow \geq$
- **Deletion policy:** A sibling can lend only if it has *strictly more* than the minimum keys (> 2 for leaves, > 1 for internals). Otherwise, **merge**.
- **Internal-key update rule:** Whenever a deleted key appears as a separator in an internal node, update it to the **new smallest key** of the corresponding right subtree/leaf.

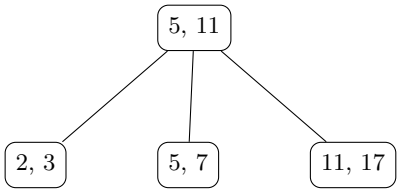
I 2, I 3, I 5: All three keys fit in a single leaf.

I 7: Leaf overflows. Split into $[2, 3]$ and $[5, 7]$; promote 5.



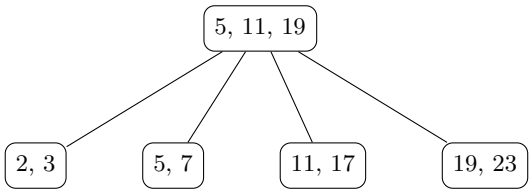
I 11: Inserted into rightmost leaf: $[5, 7, 11]$. No split needed.

I 17: Leaf $[5, 7, 11]$ overflows. Split into $[5, 7]$ and $[11, 17]$; promote 11.



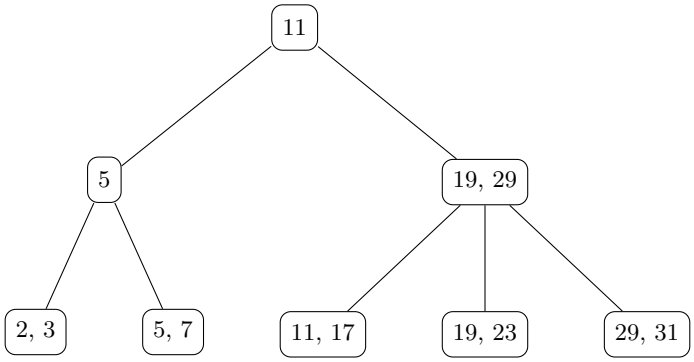
I 19: Inserted into rightmost leaf: $[11, 17, 19]$. No split.

I 23: Leaf $[11, 17, 19]$ overflows. Split into $[11, 17]$ and $[19, 23]$; promote 19.



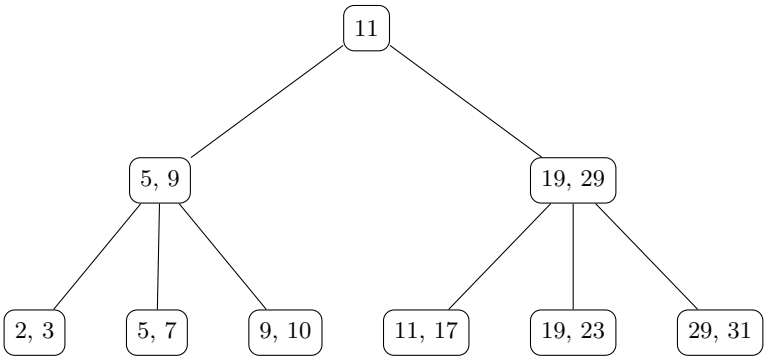
I 29: Inserted into rightmost leaf: $[19, 23, 29]$. No split.

I 31: Leaf $[19, 23, 29]$ overflows. Split leaf into $[19, 23]$ and $[29, 31]$; promote 29. Root $[5, 11, 19]$ must now absorb 29, giving 4 keys—overflow. Split the root: left child $[5]$, right child $[19, 29]$; promote 11 to a new root.

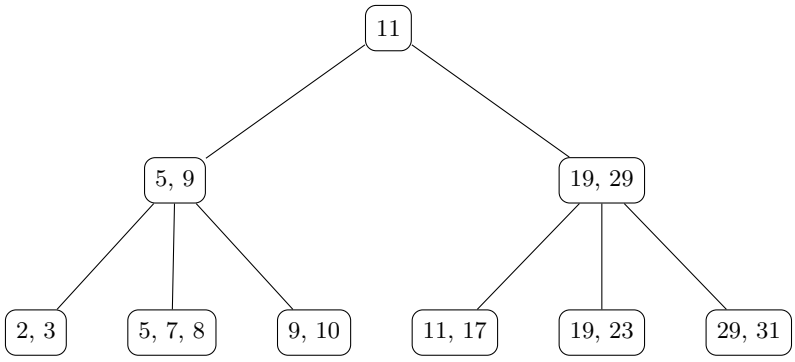


I 9: Key 9 goes into leaf $[5, 7]$, yielding $[5, 7, 9]$. No split.

I 10: Leaf $[5, 7, 9]$ overflows. Split into $[5, 7]$ and $[9, 10]$; promote 9 into parent.



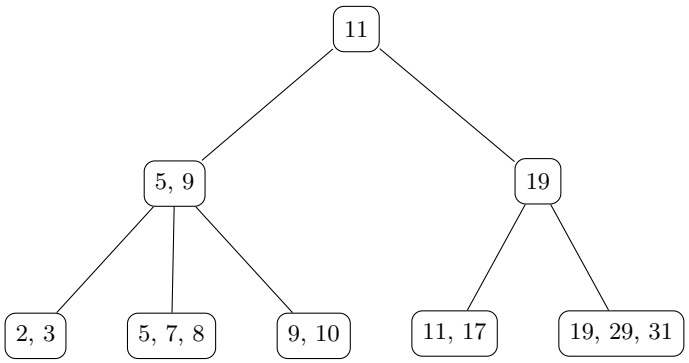
I 8: Key 8 goes into leaf $[5, 7]$, yielding $[5, 7, 8]$. No split.



D 23: Remove 23 from leaf $[19, 23]$, leaving $[19]$ (1 key, below minimum of 2). Check siblings (parent is $[19, 29]$):

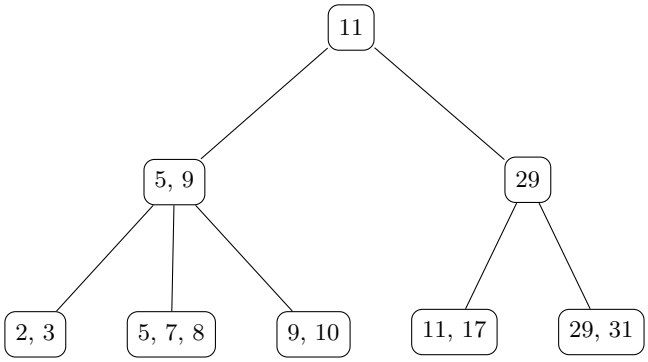
- Left sibling $[11, 17]$: has 2 keys = minimum — cannot lend.
- Right sibling $[29, 31]$: has 2 keys = minimum — cannot lend.

Merge $[19]$ with right sibling $[29, 31]$: combined leaf = $[19, 29, 31]$ (3 keys, fits). Remove separator 29 from parent $[19, 29] \rightarrow [19]$. Parent has 1 key and 2 children — valid. Key 23 does not appear in any internal node; no internal-key update needed.



D 19: Remove 19 from leaf $[19, 29, 31] \rightarrow [29, 31]$. Leaf has 2 keys $\geq$ minimum — no underflow.

Internal-key update: deleted key 19 appears as separator in parent $[19]$. Update $19 \rightarrow 29$ (new smallest key of right leaf $[29, 31]$).



D 29: Remove 29 from leaf $[29, 31] \rightarrow [31]$. Underflow (1 key $<$ minimum of 2).

Internal-key update: deleted key 29 appears as separator in parent $[29]$. Update $29 \rightarrow 31$ (new smallest key of right leaf $[31]$). Parent is now $[31]$.

Check siblings (parent is $[31]$): Left sibling $[11, 17]$: has 2 keys = minimum — cannot lend. No right sibling.

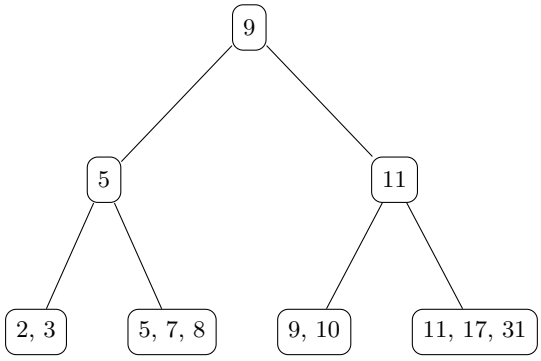
Merge [31] with left sibling $[11, 17]$: combined leaf = $[11, 17, 31]$ (3 keys, fits). Remove separator 31 from parent $[31] \to [\,]$ (empty). Parent now has 0 keys and 1 child pointer — underflow for a non-root internal node (minimum is 1 key / 2 pointers).

Propagate underflow to the internal level. The empty right child of the root has sibling $[5, 9]$. $[5, 9]$ has 2 keys / 3 pointers, strictly more than the minimum of 1 key / 2 pointers — can redistribute.

Redistribute through the root:

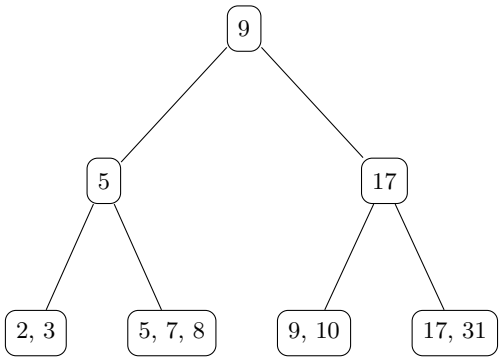
- Pull down root key 11 into the deficient (empty) right node.
- Push up the rightmost key of $[5, 9]$, which is 9, into the root.
- Transfer the rightmost child of $[5, 9]$ (leaf $[9, 10]$) to become the leftmost child of the right node.

Result: root = $[9]$; left internal = $[5]$ with children $[2, 3]$, $[5, 7, 8]$; right internal = $[11]$ with children $[9, 10]$, $[11, 17, 31]$.



D 11: Remove 11 from leaf $[11, 17, 31] \to [17, 31]$. Leaf has 2 keys $\geq$ minimum — no underflow.

Internal-key update: deleted key 11 appears as separator in right internal node [11]. Update $11 \to 17$ (new smallest key of right leaf $[17, 31]$).



Final B^+ tree: Root [9] with internal nodes [5] and [17], and leaves $[2, 3]$, $[5, 7, 8]$, $[9, 10]$, $[17, 31]$ linked left-to-right.

Answer 4

Hash function $h(x) = x \bmod 8$. Bucket capacity = 3. **Most significant bits** of the 3-bit hash value determine the bucket.

Hash values for all keys:

Key	$h(x)$ (bin)	Key	$h(x)$ (bin)	Key	$h(x)$ (bin)	Key	$h(x)$ (bin)
2	010	11	011	29	101	9	001
3	011	17	001	31	111	10	010
5	101	19	011			8	000
7	111	23	111				

Initial state: Global depth = 1. Two buckets: directory entries $0 \to A$, $1 \to B$. Both buckets have local depth 1.

I 2 (010), I 3 (011): Both have MSB = 0; go to bucket A. $A = [2, 3]$.
I 5 (101), I 7 (111): Both have MSB = 1; go to bucket B. $B = [5, 7]$.
I 11 (011): MSB = 0; bucket A becomes $[2, 3, 11]$ (full).

I 17 (001): MSB = 0; bucket A overflows. Local depth = global depth = 1, so **double the directory**. Global depth $\to$ 2.

Split A using 2 MSBs: keys 2 (01), 3 (01), 11 (01) stay in bucket A (prefix 01); key 17 (00) goes to new bucket A′ (prefix 00). Local depths of A and A′ become 2.

Directory (2 MSB)	Bucket contents
00	$A' = [17]$ (ld = 2)
01	$A = [2, 3, 11]$ (ld = 2)
10, 11	$B = [5, 7]$ (ld = 1)

I 19 (011): 2-MSB prefix = 01; bucket A is full. Local depth = 2 = global depth, so **double directory**. Global depth $\to$ 3.

Split A using 3 MSBs: 2 (010), 3 (011), 11 (011), 19 (011). Keys with prefix 010 go to A_1; prefix 011 go to A_2.

Directory (3 MSB)	Bucket contents
000, 001	$A' = [17]$ (ld = 2)
010	$A_1 = [2]$ (ld = 3)
011	$A_2 = [3, 11, 19]$ (ld = 3)
100, 101, 110, 111	$B = [5, 7]$ (ld = 1)

I 23 (111): prefix 111 $\to B$. $B = [5, 7, 23]$ (fits).

I 29 (101): prefix 101 $\to B$; bucket B overflows. Local depth 1 < global depth 3, so split **without** doubling. Split B using 2 MSBs: 5 (10), 7 (11), 23 (11), 29 (10).

Directory (3 MSB)	Bucket contents
000, 001	$A' = [17]$ (ld = 2)
010	$A_1 = [2]$ (ld = 3)
011	$A_2 = [3, 11, 19]$ (ld = 3)
100, 101	$B_1 = [5, 29]$ (ld = 2)
110, 111	$B_2 = [7, 23]$ (ld = 2)

I 31 (111): prefix 111 $\rightarrow B_2$. $B_2 = [7, 23, 31]$ (fits).
I 9 (001): prefix 001 $\rightarrow A'$. $A' = [17, 9]$ (fits).
I 10 (010): prefix 010 $\rightarrow A_1$. $A_1 = [2, 10]$ (fits).
I 8 (000): prefix 000 $\rightarrow A'$. $A' = [17, 9, 8]$ (fits).

State after all insertions:

Directory (3 MSB)	Bucket contents
000, 001	$[8, 9, 17]$ (ld = 2)
010	$[2, 10]$ (ld = 3)
011	$[3, 11, 19]$ (ld = 3)
100, 101	$[5, 29]$ (ld = 2)
110, 111	$[7, 23, 31]$ (ld = 2)

D 23: Remove from $[7, 23, 31] \rightarrow [7, 31]$.
D 19: Remove from $[3, 11, 19] \rightarrow [3, 11]$.
D 29: Remove from $[5, 29] \rightarrow [5]$.
D 11: Remove from $[3, 11] \rightarrow [3]$.

Final extendible hash structure (global depth = 3):

000, 001 $[8, 9, 17]$ (ld = 2)
010 $[2, 10]$ (ld = 3)
011 $[3]$ (ld = 3)
100, 101 $[5]$ (ld = 2)
110, 111 $[7, 31]$ (ld = 2)

Question 5: Linear Hashing

Initial setup: 2 buckets (B0, B1), $h_0(x) = x \bmod 2$, bucket capacity = 3, split pointer $p = 0$ at level 0.

Routing rule: for a key x, compute $h_0(x)$. If $h_0(x) \geq p$, the key goes to bucket $h_0(x)$. If $h_0(x) < p$, use $h_1(x) = x \bmod 4$ to determine the bucket.

When the split pointer reaches n (the number of buckets at the start of the current level), the level advances: the base hash becomes h_1, n doubles, and p resets to 0.

I 2: $h_0(2) = 0 \geq p{=}0 \rightarrow$ B0. B0: [2].
I 3: $h_0(3) = 1 \geq 0 \rightarrow$ B1. B1: [3].
I 5: $h_0(5) = 1 \rightarrow$ B1. B1: [3, 5].
I 7: $h_0(7) = 1 \rightarrow$ B1. B1: [3, 5, 7] (full).

I 11: $h_0(11) = 1 \rightarrow$ B1 (full). A new overflow block is allocated: B1:[3, 5, 7] $\rightarrow$ OV:[11].

Overflow triggers split of bucket at $p = 0$. Rehash B0 contents using $h_1(x) = x \bmod 4$: $h_1(2) = 2$, so key 2 moves to new bucket B2. B0 becomes empty. Advance $p \rightarrow 1$.

Since $p = 1$ and there were $n = 2$ original buckets at level 0, we check: $p < n$, so we remain at level 0.

B0: [ ] B1: [3, 5, 7] $\rightarrow$ OV:[11] B2: [2]
Level 0, $p = 1$, 3 buckets total.

I 17: $h_0(17) = 1 \geq p{=}1 \rightarrow$ B1. Overflow block OV has room: OV becomes [11, 17]. No new overflow block allocated, so no split.

I 19: $h_0(19) = 1 \rightarrow$ B1. OV becomes [11, 17, 19] (full). No *new* overflow block yet.

I 23: $h_0(23) = 1 \rightarrow$ B1. OV is full; allocate a second overflow block OV2:[23].

Overflow triggers split of bucket at $p = 1$. Rehash all of B1's contents (including overflow) using $h_1(x) = x \bmod 4$:

$$h_1(3) = 3,\ h_1(5) = 1,\ h_1(7) = 3,\ h_1(11) = 3,\ h_1(17) = 1,\ h_1(19) = 3,\ h_1(23) = 3$$

Keys with $h_1 = 1$ stay in B1: $\{5, 17\}$. Keys with $h_1 = 3$ go to new B3: $\{3, 7, 11, 19, 23\}$. Since B3 can hold 3, we get B3:[3, 7, 11] $\rightarrow$ OV3:[19, 23].

Advance $p \rightarrow 2$. Now $p = 2 = n = 2$, so **advance to level 1**: base hash becomes $h_1(x) = x \bmod 4$, $n = 4$, $p = 0$.

B0: [ ] B1: [5, 17]
B2: [2] B3: [3, 7, 11] $\rightarrow$ OV3:[19, 23]
Level 1, $h_1 = x \bmod 4$, $p = 0$.

I 29: $h_1(29) = 1 \geq 0 \rightarrow$ B1. B1 becomes $[5, 17, 29]$. No overflow.

I 31: $h_1(31) = 3 \rightarrow$ B3. OV3 has room: OV3 becomes $[19, 23, 31]$. No new overflow.

I 9: $h_1(9) = 1 \rightarrow$ B1 (full). Allocate new overflow: B1:$[5, 17, 29] \rightarrow$ OV4:[9].

Split bucket at $p = 0$. Rehash B0 using $h_2(x) = x \bmod 8$: B0 is empty, so nothing moves. Create B4 (empty). Advance $p \rightarrow 1$.

B0: [ ] B1: $[5, 17, 29] \rightarrow$ OV4:[9] B2: [2]
B3: $[3, 7, 11] \rightarrow$ OV3:$[19, 23, 31]$ B4: [ ]
Level 1, $p = 1$. For bucket 0: use $h_2 = x \bmod 8$.

I 10: $h_1(10) = 2 \geq p$=1 $\rightarrow$ B2. B2 becomes $[2, 10]$. No overflow.

I 8: $h_1(8) = 0 < p$=1, so use $h_2(8) = 8 \bmod 8 = 0 \rightarrow$ B0. B0 becomes [8]. No overflow.

Deletions:

D 23: Remove 23 from OV3. OV3 becomes $[19, 31]$.

D 19: Remove 19 from OV3. OV3 becomes [31].

D 29: Remove 29 from B1. B1 becomes $[5, 17]$. Overflow OV4:[9] remains unchanged.

D 11: Remove 11 from B3. B3 becomes $[3, 7]$. Overflow OV3:[31] remains unchanged.

Final linear hash structure (Level 1, $n = 4$, $p = 1$):
$h_1 = x \bmod 4$ for buckets $\geq p$; $h_2 = x \bmod 8$ for buckets $< p$ (i.e., bucket 0).

B0: [8] B1: $[5, 17] \rightarrow$ OV4: [9]
B2: $[2, 10]$ B3: $[3, 7] \rightarrow$ OV3: [31]
B4: [ ] (empty)